Process Concept and the Process Control Block

Lecture Notes — Lecture 5
What Is a Running Program?

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

5.1 One File, Two Runs

Compile `counter.c` into a single executable file, `./counter`, and run it in two terminals at the same time. Each copy counts upward from 1 at its own pace. There is *one* file on disk, yet the operating system is clearly running *two* things. The question that organizes this chapter is: what exactly are those two things, and what must the kernel record about each?

Three observations frame the answer. First, the file on disk does not change while it runs; it is just bytes. Second, each run has its own counter value, its own position in the code, and its own stack. Third, the OS must remember all of this for each run separately. We will call the two runs `P1` and `P2` and follow them through the chapter, asking at every step what the kernel must record about each.

The chapter answers four questions in turn: **what is a process** (a program in execution, not the program file); **what does a process look like in memory** (text, data, heap, and stack); **what states can a process be in** (new, ready, running, waiting, terminated); and **how does the kernel remember a process** (the process control block, or PCB). The previous lecture organized the kernel itself; this one names the unit of work the kernel manages, and the next lecture watches that unit move between queues.

5.2 Program versus Process

Reading: Silberschatz et al., Ch. 3 §3.1, pp. 101–103; Stallings, Ch. 3 §3.1, p. 108.

A useful image: a program is a recipe written down; a process is somebody cooking it — with the oven on, the ingredients half-mixed, and a finger on line 7. Table 5.1 makes the contrast precise.

Program	**Process**
A file on disk (an executable)	A program in execution
Passive: a list of instructions	*Active*: has a program counter and live resources
One copy, however many runs	One per run, each independent
Exists until deleted	Created, runs, and terminates

Table 5.1: Program versus process (Silberschatz et al., Ch. 3 §3.1, pp. 101–103).

A program becomes a process when its executable file is **loaded into memory**. Two processes may run the same program; they are nevertheless two separate execution sequences.

Definition (Process). *A process is a program in execution: the unit of work in a modern time-sharing system. It consists of the program code together with its current activity — the value of the program counter, the contents of the processor's registers, and its memory.*

Batch systems called the same entity a **job**, and the textbook uses *job* and *process* almost interchangeably (Silberschatz, p. 101). Stallings (§3.1, p. 108) lists equivalent views: a program in execution, an instance of a program running on a computer, and the entity that can be assigned to and executed on a processor.

Example 5.1 (`./counter` launched twice)**.** *1. Terminal A runs `./counter`. The OS creates `P1`: it loads the code, allocates a stack, and sets the program counter to the entry point.*

2. *Terminal B runs `./counter`. The OS creates `P2` from the* same *file, with a* separate *stack and data.*

3. *After a few seconds `P1` has printed 812 and `P2` has printed 305.*

4. *The variable `count` therefore has two values at once — one in each process's data.*

5. *Both program counters may point into the same loop, at different instructions.*

Conclusion: *same text section, different activity; the OS tracks `P1` and `P2` as two processes.*

The counting criterion is not what is visible on screen. If you open three windows of the same browser, the number of *windows* does not decide how many processes exist; the number of *execution sequences the OS tracks* does. One process can draw many windows, and one program can also start several processes. The question to put to the OS is: how many PCBs exist for this program?

5.3 A Process in Memory

Reading: Silberschatz et al., Ch. 3 §3.1, pp. 101–102; Stallings, Ch. 3 §3.3, p. 126.

Before the kernel can record a process, the process needs a place to live. Its address space has four regions, each with a job (Figure 5.1). The **text** section holds the program code. The **data** section holds global variables. The **heap** holds memory allocated at run time and grows upward. The **stack** holds function parameters, return addresses, and local variables, and grows downward. Between heap and stack lies free space into which both can grow.

Example 5.2 (Where does each variable live?)**.** *Consider this fragment of `counter.c`:*

```
int count = 0;  /* (a) */
int main(void) {
  int step = 1;  /* (b) */
  int *log =
    malloc(64);  /* (c) */
  while (1) {
    count += step; /* (d) */
    print(count);
  }
}
```

Concepts That Click

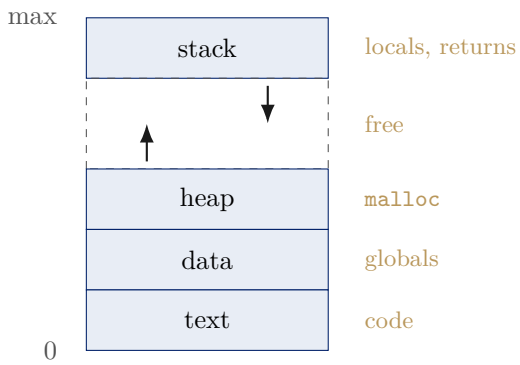


Figure 5.1: The four regions of a process's address space, from address 0 (text) up to the maximum address (stack); the heap grows up and the stack grows down into the free band — cf. Silberschatz et al., Ch. 3 §3.1, pp. 101–102 [2].

*(a) `count` is a global, so it lives in **data**. (b) `step` is a local variable of `main`, so it lives on the **stack**. (c) The 64 bytes that `log` points to were allocated at run time, so they are on the **heap**; the pointer variable `log` itself is a local and lives on the **stack**. (d) The instructions of the loop are code, in **text**. For `P1` and `P2` the text is the same, while data, heap, and stack are separate: `count` is 812 in one and 305 in the other.*

Stallings (§3.3, p. 126) calls the whole collection a **process image**: the user program (text), user data (data and heap), the stack, and the process's **control block**, the kernel's record of it. The first three are what the process uses; the last is what the *kernel* uses to manage it. Put another way, the memory regions describe what the process *is doing*, while the control block describes what the kernel *knows about it*.

5.4 Process States

Reading: Silberschatz et al., Ch. 3 §3.1, p. 103; Stallings, Ch. 3 §3.2, p. 111.

A process is not always running. Most of the time it is waiting — for the CPU or for the outside world — and the kernel must know which. Silberschatz names five states (Table 5.2).

State	Meaning
new	The process is being created.
ready	The process is waiting to be assigned to a processor.
running	Instructions are being executed.
waiting	The process is waiting for some event (I/O completion, a signal). Stallings: *blocked*.
terminated	The process has finished execution. Stallings: *exit*.

Table 5.2: The five process states.

Definition (Counting rule). *On one processor, at most one process is running at any instant; many may be ready or waiting. With k CPUs, at most k processes run.*

Figure 5.2 shows the state diagram. It has exactly six transitions, and each is caused by a specific event, listed in Table 5.3.

Concepts That Click

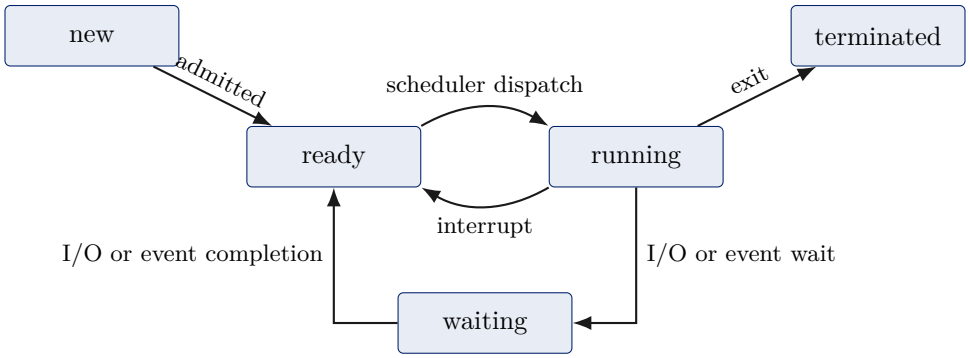


Figure 5.2: The process-state diagram, after Silberschatz et al., Ch. 3 §3.1, p. 103 [2].

Transition	**Caused by**
new → ready	the OS admits the process (resources allocated)
ready → running	the scheduler dispatches it to the CPU
running → ready	an interrupt, e.g. the timer: its time is up
running → waiting	it requests I/O or waits for an event (a blocking system call such as `read`)
waiting → ready	the I/O completes or the event occurs
running → terminated	it calls `exit` (or is killed)

Table 5.3: The cause of each transition.

Two arrows are notable for being absent. There is no waiting → running arrow: a waiting process must first become ready. And there is no ready → waiting arrow: a ready process is not executing, so it cannot request I/O. The first absence deserves a reason. When a waiting process receives its I/O, another process is probably running on the CPU right now; only the scheduler decides who runs next, and the I/O completion only makes the process *eligible*. Keeping "eligible" and "chosen" separate is what lets the OS apply a scheduling policy at all.

Example 5.3 (`P1` reads a key)**.** 1. *`P1` is created:* new.

2. *The OS admits it:* ready.

3. *The scheduler dispatches it:* running*; it increments* `count`.

4. *The timer interrupt fires: back to* ready*;* `P2` *runs.*

5. `P1` *is dispatched again:* running. *It calls* `read` *on the keyboard:* waiting.

6. *The user presses a key; the interrupt completes the I/O:* ready.

7. `P1` *is dispatched:* running. *It calls* `exit`*:* terminated.

The trace visits seven states and takes six transitions. The blocking `read` *is the system call of Lecture 3 seen from the process's side.*

Example 5.4 (Counting states on one CPU)**.** *Five processes exist.* `P1` *is running;* `P2` *and* `P3` *wait for the disk;* `P4` *and* `P5` *are ready.* `P1`*'s time slice expires. The interrupt moves* `P1` *to ready, and the scheduler picks* `P4`*, which becomes running. Afterwards: running 1 (*`P4`*); ready 2 (*`P5`*,* `P1`*); waiting 2 (*`P2`*,* `P3`*). The total is still five: transitions move processes, they do not create them.*

Concepts That Click

Stallings (§3.2, p. 111) reaches the same five states from a simpler start. His **two-state** model has only *running* and *not running*. It is too coarse: a not-running process might be ready to go or blocked on I/O, and dispatching a blocked process wastes the CPU. Splitting *not running* into *ready* and *blocked*, and adding *new* and *exit*, gives the **five-state** model. Stallings later adds *suspended* states for swapped-out processes (a seven-state model); we return to these with the medium-term scheduler.

5.5 The Process Control Block

Reading: Silberschatz et al., Ch. 3 §3.1, pp. 103–104; Stallings, Ch. 3 §3.3, p. 126; Bovet & Cesati, Ch. 3, pp. 79–130.

When `P1` stops running, everything it was doing must be written down somewhere — or it can never be resumed. That somewhere is the PCB.

Definition (Process control block). *The process control block (PCB), also called a task control block, is the kernel data structure that represents one process. It is the repository for any information that may vary from process to process.*

There is one PCB per process: `P1` and `P2` have two PCBs even though they share one program file. The PCB lives in *kernel* memory, and the process cannot read or write it directly. Creating a process means creating a PCB; terminating it means eventually freeing the PCB. Figure 5.3 shows its principal fields.

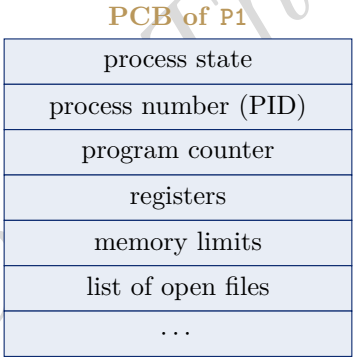


Figure 5.3: The fields of a PCB, after Silberschatz et al., Ch. 3 §3.1, p. 103 [2].

The *state* is one of new, ready, running, waiting, terminated. The *PID* is the unique process number. The *program counter* holds the address of the next instruction to execute. The *registers* are the accumulators, index registers, stack pointer, general-purpose registers, and condition codes. The *memory limits* say where the process's regions are, and the *open files* say what it is using. Silberschatz (pp. 103–104) groups the information into seven categories (Table 5.4).

Why save the program counter and registers in particular? When an interrupt takes the CPU away from `P1`, the CPU's registers still hold `P1`'s values, and `P2` is about to overwrite them. So the kernel first copies `P1`'s PC and registers into `P1`'s PCB. Later it copies them back, and `P1` continues as if nothing had happened. State information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward; the PCB is what makes interruption safe. Saving one process's state and loading another's is the **context switch**, the topic of the next lecture.

Example 5.5 (Two PCBs, one program)**.** *At one instant the two `./counter` processes look like this:*

Concepts That Click

Category	Examples
Process state	new, ready, running, waiting, halted
Program counter	address of the next instruction
CPU registers	accumulators, index registers, stack pointers, general-purpose registers, condition codes
CPU-scheduling information	priority, pointers to scheduling queues
Memory-management information	base and limit registers, page tables or segment tables
Accounting information	CPU time used, time limits, job or process numbers
I/O status information	I/O devices allocated, list of open files

Table 5.4: The seven categories of PCB information (Silberschatz et al., Ch. 3 §3.1, pp. 103–104).

Field	***PCB of `P1`***	***PCB of `P2`***
state	*waiting (keyboard)*	*running*
PID	*4120*	*4127*
program counter	*inside `read` wrapper*	*inside the loop*
`count` (in data)	*812*	*305*
open files	*terminal A*	*terminal B*
text section	*shared: same `./counter` code*	

Every row that differs is per-process information — exactly what the PCB exists to hold. The PIDs are illustrative; the OS picks them.

Stallings (§3.3, p. 126) groups the same content by purpose into three parts. *Process identification*: PID, parent's PID, user ID. *Processor state information*: user-visible registers, control and status registers (PC, flags), and stack pointers. *Process control information*: scheduling and state, links to other processes, privileges, and memory and resource ownership. The processor-state part is what a context switch saves and restores.

In Linux the PCB is the **process descriptor**, a C structure of type `task_struct` (Bovet & Cesati, Ch. 3, pp. 79–130 [1]). It holds the state, the PID, scheduling information, a pointer to the memory descriptor, the open-file table, parent and child links, and much more. Its `state` field records values such as `TASK_RUNNING`, `TASK_INTERRUPTIBLE`, and `TASK_UNINTERRUPTIBLE`, and the kernel links all descriptors into a list so it can walk every process. Note the naming: Linux uses `TASK_RUNNING` for both ready and running, so the textbook's two states map to one flag plus the question "is it on a CPU right now?"

Why can the PCB not be stored in the process's own stack or heap, where it would be cheaper to reach? Because the process could then overwrite its own state, PID, or memory limits and break protection; and because the kernel must read the PCB even when the process is not running and its memory may be swapped out. So PCBs live in kernel memory, reachable only in kernel mode.

5.6 Threads: A Preview

Reading: Silberschatz et al., Ch. 3 §3.1, pp. 104–105; threads in full in Ch. 4.

A process is its address space plus its PCB. One more refinement completes the picture. The model so far assumes that a process performs a *single thread of execution* — one program counter.

Concepts That Click

Modern systems let a process have **multiple threads**, each with its own PC and registers, sharing the same address space; the PCB is then extended with per-thread information. A word processor with one thread cannot type characters and run the spell checker at the same time; with two threads it can.

5.7 Misconceptions and Summary

Four errors recur. "A process is the program file" — no: a process is a program *in execution*. "Two runs of one program share one PCB" — no: there is one PCB per process, even for the same program. "I/O completion puts a process back on the CPU" — no: it goes to *ready*, and the scheduler decides. "Many processes can be running on one CPU" — no: at most one at a time per CPU; the others are ready or waiting.

In summary: a **process** is a program in execution; the program is passive and the process is active. Its address space has **text**, **data**, **heap**, and **stack**, and two processes running one program share text only. It moves among five **states** — new, ready, running, waiting, terminated — through six legal transitions, each caused by an event. The **PCB** records everything per-process: state, PC, registers, scheduling, memory, accounting, and I/O status. Saving the PC and registers into the PCB is what makes interruption and resumption possible.

5.8 Looking Ahead and Further Reading

The next lecture examines the queues that hold PCBs, the long-, short-, and medium-term schedulers that move them, and the context switch that saves one PCB and loads another. Its reading is Silberschatz et al., Ch. 3 §3.2 [2]; Stallings, Ch. 3 §3.2 and §3.4 [3]; and Bovet & Cesati, Ch. 3 [1]. For this chapter the page-cited sources are:

- Silberschatz, Galvin, and Gagne, *Operating System Concepts*, 8th ed. — Ch. 3 §3.1 (the process, p. 101; process state, p. 103; process control block, p. 103; threads, pp. 104–105).
- Stallings, *Operating Systems: Internals and Design Principles*, 6th ed. — Ch. 3 §3.1 (p. 108), §3.2 (p. 111), §3.3 (p. 126).
- Bovet and Cesati, *Understanding the Linux Kernel*, 3rd ed. — Ch. 3 (pp. 79–130), for the concrete `task_struct`.

All are page-verified through each book's `index.md` in the course supporting-books directory:

`master_supporting_docs/CS502/supporting_books/`

Self-Check

Exercises

1. State, for each of the following, which region of the address space holds it: a `static` global array, the return address of a function call, a buffer returned by `malloc`, the machine code of `main`.
2. A process goes: created, admitted, dispatched, timer interrupt, dispatched, requests disk I/O, I/O completes, dispatched, exits. List the states it visits and count the transitions.

Concepts That Click

3. On a dual-core machine, eight processes exist: three are blocked on I/O. What is the maximum number running, and the minimum number ready?

4. Explain why a waiting process cannot move directly to running.

5. Which PCB category holds each of: the page-table pointer, the priority, the CPU time used, the list of open files?

Solutions

1. Global array: data. Return address: stack. `malloc` buffer: heap. Machine code of `main`: text.

2. States: new, ready, running, ready, running, waiting, ready, running, terminated — nine states visited (counting repeats), and eight transitions: admitted, dispatch, interrupt, dispatch, I/O wait, I/O completion, dispatch, exit.

3. At most $k = 2$ are running. The five non-blocked processes are either running or ready, so with two running at least $5 - 2 = 3$ are ready.

4. I/O completion only makes the process eligible; another process is probably running, and only the scheduler chooses which ready process runs. Separating eligibility from selection is what allows a scheduling policy.

5. Page-table pointer: memory-management information. Priority: CPU-scheduling information. CPU time used: accounting information. Open files: I/O status information.

References

[1] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition, 2006.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[3] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.

Concepts That Click